

Architecture Explainer — One-Page Plan

The problem

Writing and maintaining architecture docs, diagrams, and onboarding material is hard and slow. Companies like Mozilla, Chrome, and Microsoft (PowerShell) run dedicated teams just for this. Most engineering orgs don't have that luxury, so docs go stale or never get written — and new engineers take longer to become productive.

The idea

Connect a code repo (like how Slack or Codex apps connect). The tool reads and understands the codebase, then automatically generates:

- Architecture documentation
- Diagrams (visual maps of how the system fits together)
- Onboarding guides for new engineers

Developers use these docs to understand the system and contribute faster, without waiting on a human writer.

How it works

1. **Connect** — A GitHub/GitLab App links the repo (Slack app can be added later for Q&A).
2. **Ingest** — The code is parsed and broken into meaningful chunks.
3. **Index** — Each chunk is embedded and stored in a vector database so the system can search by meaning, not just keywords.
4. **Understand** — An LLM reviews related chunks and writes plain-language explanations of what each part does and how pieces connect.
5. **Diagram** — Those explanations are turned into diagrams.
6. **Publish** — Docs and diagrams are output as a docs site, or committed back into the repo so they stay versioned with the code.

Tech choices (and why)

Layer	v1 choice	Later option	Why
Vector search	pgvector	Qdrant	pgvector is simple to run (just Postgres) and fast enough at moderate scale. Move to Qdrant only if performance/filtering demands it.

Layer	v1 choice	Later option	Why
Diagrams	Mermaid	PlantUML / Graphviz	Mermaid renders natively in GitHub/GitLab and most docs sites — zero extra tooling. Other formats add diagram types later.
Integrations	Repo connector (GitHub App)	Slack, Linear, Notion	The repo connection is the core loop. Everything else is additive.

Who it's for

- **Now:** engineering teams that want architecture docs and onboarding material without a dedicated docs team.
- **Later:** companies with large codebases who need this at scale, potentially self-hosted for data privacy.

What v1 should NOT try to do

- Multiple languages at once — pick one to start.
- Slack Q&A — nice to have, not core.
- PlantUML/Graphviz — Mermaid only for now.
- Perfect diagrams — a good first draft that a human can polish beats a fully automated but shallow one.

Success looks like

A new engineer can read the generated architecture doc and diagram for an unfamiliar repo and explain, in their own words, how the main pieces connect — without reading the code first.